

프롬프트 하나로 데이터 분석 시키기: LangChain으로 배우는 AI 분석

LangChain - Prompt, Memory

김종영



RetryWithErrorOutputParser

●작동 방식

- 초기 파싱 시도: LLM 응답을 기본 파서로 파싱
- 오류 감지: 파싱 실패 시, **RetryWithErrorOutputParser**가 오류를 포착
- 재시도 프롬프트 생성: 잘못된 출력과 오류 메시지를 포함하여,
LLM에게 올바른 형식으로 재응답하도록 지시하는 새로운 프롬프트를 생성
- 자동 재시도: LLM이 새로운 프롬프트를 바탕으로 교정된 응답을 생성하고, 파싱 성공 시 최종 결과를 반환

●핵심 구성 요소

- base_parser**: 최종 출력을 파싱할 기본 파서 (예: **SimpleJsonOutputParser**).
- llm**: 재시도 요청을 처리하는 데 사용될 언어 모델.
- get_format_instructions()**: 프롬프트에 포함하여 LLM에게 원하는 출력 형식을 명확히 알려주는 지침을 제공

OutputFixingParser

●작동 방식

- 기본 파서 설정: 유효한 출력을 파싱할 기본 파서를 정의함.
- OutputFixingParser 초기화: 기본 파서와 교정에 사용될 LLM을 인자로 받아 OutputFixingParser를 생성함.
- 교정 프롬프트 생성: 파싱에 실패하면, OutputFixingParser는 LLM에게 "이전 응답이 잘못되었으니 이 부분을 수정하여 올바른 형식으로 만들어달라"고 지시하는 새로운 프롬프트를 생성함. 이때, 잘못된 출력과 오류 메시지가 프롬프트에 포함
- 자동 교정: LLM은 이 프롬프트를 바탕으로 오류를 수정하고, OutputFixingParser는 교정된 출력을 최종적으로 파싱하여 반환

●핵심 구성 요소

- parser: 최종 출력을 파싱할 기본 파서 (예: SimpleJsonOutputParser).
- llm: 오류를 교정하는 데 사용될 언어 모델.
- 교정 프롬프트 (내부 동작): 잘못된 출력과 오류 메시지를 사용해서 교정

ChatPromptTemplate

●작동 방식

—메시지 템플릿 정의: **SystemMessagePromptTemplate**, **HumanMessagePromptTemplate**, **AIMessagePromptTemplate** 등 특정 역할을 가진 메시지 템플릿을 정의함. 각 템플릿은 변수 (variables)를 포함할 수 있음.

—**ChatPromptTemplate** 구성: 정의된 메시지 템플릿들을 순서대로 리스트에 담아 **ChatPromptTemplate** 객체를 생성함.

—변수 바인딩: **format_messages()** 메서드에 변수 값을 전달하여 최종적인 메시지 객체 리스트를 생성함. 이 리스트는 LLM에 바로 전달될 수 있는 형태로, 각 메시지의 역할(role)이 명확히 구분

●핵심 구성 요소

—**SystemMessagePromptTemplate**: LLM에게 특정 행동이나 역할을 부여하는 지침을 제공

—**HumanMessagePromptTemplate**: 사용자(인간)의 질문이나 발언을 담는 템플릿.

—**AIMessagePromptTemplate**: LLM(AI)의 이전 답변을 담는 템플릿으로, 대화 이력을 구성할 때 사용

RunnableWithMessageHistory

●작동 방식

—세션 관리: **RunnableWithMessageHistory**는 세션 ID를 사용해 각 대화를 고유하게 식별. 이를 통해

여러 사용자의 대화 이력을 분리하여 관리

—대화 이력 저장: 사용자가 메시지를 보내면, 이 객체는 지정된 대화 이력 저장소

(**ChatMessageHistory**

등)에서 해당 세션의 이전 대화를 불러옴.

—프롬프트 구성: 불러온 대화 이력과 현재의 사용자 입력을 결합하여 LLM에게 전달할 최종 프롬프트를 구성함.

—자동 업데이트: LLM이 응답을 생성하면, 이 응답을 다시 대화 이력 저장소에 추가하여 다음 대화를 위한 문맥을 갱신함.

●핵심 구성 요소

—**runnable**: 대화 이력이 적용될 LLM 체인 또는 Runnable 객체.

—**get_session_history**: 세션 ID를 받아 해당 ID에 맞는 **BaseChatMessageHistory** 객체를 반환하는 함수.

—**input_messages_key**: 체인 내에서 사용자 입력 메시지에 해당하는 키(key) 이름.



FewShotChatMessagePromptTemplate

- 작동 방식

- 예시 정의: 먼저, **HumanMessage**와 **AIMessage** 객체로 구성된 예시 대화 리스트를 생성.
- 예시 포맷터 (**Example Formatter**) 설정: 선택된 예시를 **ChatPromptTemplate** 내에서 사용할 수 있는 형식으로 포맷팅
- FewShotChatMessagePromptTemplate** 구성: 예시와 **example_prompt**를 결합하여 최종 템플릿을 만듦. 이 템플릿은 사용자의 실제 입력과 함께 예시를 결합하여 LLM에게 전달할 최종 프롬프트를 완성

- 핵심 구성 요소

- example_selector**: 프롬프트에 포함될 예시를 선택하는 로직을 담당
- example_prompt**: 선택된 예시를 어떤 형식으로 LLM에게 보여줄지 정의함. 주로 **ChatPromptTemplate**을 사용함.
- input_variables**: 예시가 아닌 실제 사용자 입력에 사용될 변수명.

FileChatMessageHistory

●작동 방식

—파일 경로 지정: **FileChatMessageHistory** 객체를 초기화할 때, 대화 내용을 저장할 파일의 경로를 지정

—메시지 추가: **add_user_message()** 또는 **add_ai_message()** 메서드를 호출하면, 해당 메시지가 메모리 내 리스트에 추가됨과 동시에, 지정된 파일에 **JSON** 형식으로 자동 저장됨.

—메시지 불러오기: 객체가 초기화될 때, 지정된 경로에 파일이 이미 존재하면, 해당 파일의 내용을 읽어와 대화 이력 객체를 자동으로 복원함.

●핵심 구성 요소

—**file_path**: 대화 이력을 저장하고 불러올 파일의 경로. 이 경로는 문자열 형태로 제공됨.

※ **RunnableWithMessageHistory**의 파일 시스템 기반 저장소 역할을 담당할 수 있음